Project report

Machine Learning

NLP Challenge: Polish Peruvian Assistant

Team members:

Pierina Lopez r0913865

Julia Drygalska r0912428

Introduction 1
Code explanation 2
Building the recipe vector database (ingest.py) 2
Language Models and Chef Agents (agents.py) 6
Retrieval, Unit Conversion and Nutrition tools (tools.py) 10
Orchestration Graph and Conversation Flow (graph.py) 14
Streamlit frontend and Chat Interface (app_streamlit.py) 21
Metrics 22
Conclusion 23
AI use 24

Introduction

In this project we have built an intelligent multi-agent cooking assistant that can answer questions about Polish and Peruvian recipes, explain techniques and adapt its answers to the user's language (Polish, Spanish or English).

The goal was not to just create a tool that only returns generic recipes, but uses a private recipe knowledge base, can make a fusion between cuisines, reasons about nutrition, and keeps short-term conversation context for more natural interactions.

To achieve this, we combined several technologies:

- **ChromaDB + embeddings** for storing and searching our own recipe collection (including PDFs in multiple languages)
- **LangChain + LangGraph** to orchestrate multiple agents (Polish Chef, Peruvian chef, fusion chef, nutrition agent, quizmaster and culinary historian)
- **Local LLM via Ollama** for generating answers, explanations and quizzes
- **Custom RAG pipeline** for retrieving relevant information and passing it as context to the agents
- **Streamlit** for an interactive chat-style web interface

Code explanation

Building the recipe vector database (ingest.py)

The script prepares the knowledge base for the assistant. It loads the recipes and notes from text/ Markdown files and PDFs, splits them into smaller chunks and then they are stored in ChromaDB vector store so they can be retrieved later on.

In the app, this module is used in 2 ways: from the command line, for the initial setup by running *python ingest.py --reset*, it builds the database from all files in the *data/ and data_pdfs/*. The second way is from the web interface. When the user uploads a new recipe through the UI, the app saves those files to *data_pdfs/* folder and then runs *run_ingest()* from this file, so the assistant can be provided with new knowledge directly from the browser.

Our project uses a .env file to configure external services and runtime options. The applications load these values with *load_dotenv()* in the code.

The conversational logic of all agents is powered by a local Llama. 3.2 3B model that runs locally through Ollama, while cloud models are disabled in our setup.

Our project can work offline, and it fits well with the idea of a personal learning tool that we can run on our own laptops.

We chose a 3B model even though it's smaller than big cloud models like GPT-4, so sometimes the language (especially in Polish and Spanish) is a bit less polished. To balance this out, we added several safeguards: strong system prompts that describe the desired style, automatic language detection, and a RAG layer that feeds the model real recipes from uploaded PDFs. In practice, this setup gives a good compromise between privacy, speed and answer quality.

```
# === LLM selection ===
USE_OLLAMA=true
OLLAMA_MODEL=llama3.2:3b
USE_OPENAI_LLM=false
USE_OPENAI_EMBEDDINGS=false


OPENAI_API_KE[redacted]
MODEL=gpt-4o-mini

# === Nutrition (Edamam) ===
USE_NUTRITION_API=true
EDAMAM_APP_ID=55864ea6
EDAMAM_APP_KEY=6cd8976a212c9e367f400ca3547e6730
```

```python
DATA_DIR = "data"               # .md / .txt
DATA_PDF_DIR = "data_pdfs"      # .pdf (PL / ES / EN etc.)
PERSIST_DIR = "memory/chroma_db"
COLLECTION = "recipes"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 120
ALLOWED_EXTS = (".md", ".txt")

load_dotenv()
```

```python
def _embedder():
    """Return the embedding model used for BOTH ingest and retrieval."""
    from langchain_huggingface import HuggingFaceEmbeddings
    return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

This function's purpose is to select and initialize the embedding model used to convert text chunks to vectors. It returns a *HuggingFaceEmbeddings* instance using the local model.

```python
def load_docs(path: str = DATA_DIR) -> List[Dict]:
    """Load .md / .txt documents from DATA_DIR."""
    files = glob.glob(os.path.join(path, "**", "*.*"), recursive=True)
    docs: List[Dict] = []
    for fp in files:
        if fp.lower().endswith(ALLOWED_EXTS):
            with open(fp, "r", encoding="utf-8", errors="ignore") as f:
                txt = f.read().strip()
                if txt:
                    docs.append({"text": txt, "source": fp})
    return docs
```

This code loads all the Markdown and text files from a directory to the memory. So basically, it scans the directory path for all files, filters only by the extension allowed, opens each file with UTF-8 encoding, reads the content, strips white space and discards empty files. For each valid file, it then creates a document dictionary.

```python
def load_pdfs(path: str = DATA_PDF_DIR) -> List[Dict]:
    """Load PDF documents from DATA_PDF_DIR (any language)."""
    files = glob.glob(os.path.join(path, "**", "*.pdf"), recursive=True)
    docs: List[Dict] = []
    for fp in files:
        try:
            reader = PdfReader(fp)
        except Exception:
            # Skip broken / unreadable PDFs silently
            continue

        pages_text = []
        for page in reader.pages:
            txt = page.extract_text() or ""
            if txt.strip():
                pages_text.append(txt)

        full_text = "\n\n".join(pages_text).strip()
        if full_text:
            docs.append({"text": full_text, "source": fp})
    return docs
```

This part of code loads the PDFs from a directory and extracts their textual content. In essence it scans the path for the .pdf extension, for each of them it attempts to open them using *PdfReader*, if it fails the PDF is skipped. Then it iterates over all the pages and calls page *.extract_text()*. Then it collects all the non-empty page *texts* and concatenates them into a single *full_text* string, then it returns a list of document dictionaries in the same format as *load_docs*.

```python
def make_chunks(docs: List[Dict]):
    """Split documents into overlapping chunks for vector search."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
    )
    texts, metas, ids = [], [], []
    for d in docs:
        for chunk in splitter.split_text(d["text"]):
            texts.append(chunk)
            metas.append({"source": d["source"]})
            ids.append(
                hashlib.sha1((d["source"] + "||" + chunk).encode()).hexdigest()
            )
    return texts, metas, ids
```

This code splits long documents into smaller, overlapping text chunks that are suitable for vector search and LLM context windows. First, it creates a text splitter that is configured with a target chunk size (800) and an overlap (120). Then for each document in docs it applies the splitter to get a list of chunks. For each chunk it appends the chunk string to *texts*, appends a metadata dictionary to metas, and generates an ID by hashing the combination of source and chunk text and appends it to IDs.

```python
def build_store(texts, metas, ids, embedder, reset: bool):
    """
    Create or update the Chroma collection on disk.

    If reset=True, the existing persist directory is removed first.
    """
    if reset and os.path.isdir(PERSIST_DIR):
        shutil.rmtree(PERSIST_DIR)
    os.makedirs(PERSIST_DIR, exist_ok=True)

    vs = Chroma(
        collection_name=COLLECTION,
        embedding_function=embedder,
        persist_directory=PERSIST_DIR,
    )
    if texts:
        vs.add_texts(texts=texts, metadatas=metas, ids=ids)
        try:
            vs.persist()
        except AttributeError:
            pass
```

This part of the code creates and updates the Chroma vector database with the provided text chunks and metadata. If reset is set to *True* and persistence directory exists, the directory is deleted to guarantee a clean rebuild. Then initializes Chroma collection. If there are any *texts* it inserts all chunks.

```python
def run_ingest(reset: bool = False, backend: str = "local") -> Dict:
    """
    Ingest all .md/.txt from DATA_DIR and all .pdf from DATA_PDF_DIR into Chroma.

    This function is safe to call from Streamlit (frontend) or from tests.
    It returns a small stats dict with information about what was ingested.
    """
    embedder = get_embedder(backend)

    docs_txt = load_docs()
    docs_pdf = load_pdfs()
    docs = docs_txt + docs_pdf

    if not docs:
        return {
            "backend": backend,
            "n_txt": 0,
            "n_pdf": 0,
            "n_chunks": 0,
            "message": f"No documents found in '{DATA_DIR}' or '{DATA_PDF_DIR}'. "
                       f"Add .md/.txt/.pdf files and retry.",
        }

    texts, metas, ids = make_chunks(docs)
    build_store(texts, metas, ids, embedder, reset=reset)

    return {
        "backend": backend,
        "n_txt": len(docs_txt),
        "n_pdf": len(docs_pdf),
        "n_chunks": len(texts),
        "persist_dir": os.path.abspath(PERSIST_DIR),
        "collection": COLLECTION,
    }
```

This part orchestrates the full ingestion pipeline and provides a simple API for both CLI and frontend usage. First, it chooses the embedding, loads the documents, and merges them into a single *docs* list. If there are no documents, it returns a statistics dictionary and a message. Otherwise, it calls *make_chunks()* to create text chunks, metadata, and IDs. Calls *build_store()* to create/update the Chroma collection. Then, returns a dictionary with ingestion statistics, including selected backend, number of txt and PDF files, total number of chunks, persistence directory path, and collection name.

```python
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--reset", action="store_true", help="Delete and rebuild DB.")
    parser.add_argument("--local", action="store_true", help="Use local embeddings.")
    args = parser.parse_args()

    # Pick backend (default: local unless --openai)
    backend = "openai" if args.openai else "local"
    if args.local:
        backend = "local"

    stats = run_ingest(reset=args.reset, backend=backend)

    if stats["n_chunks"] == 0:
        print(stats["message"])
        return

    print(
        f"Backend: {stats['backend']} | "
        f"Text files: {stats['n_txt']} | "
        f"PDF files: {stats['n_pdf']} | "
        f"Chunks: {stats['n_chunks']}"
    )
    print(
        f"Persisted to: {stats['persist_dir']} "
        f"(collection: {stats['collection']})"
    )


if __name__ == "__main__":
    main()
```

This provides a command-line interface for manually building or rebuilding the vector database.

Language Models and Chef Agents (agents.py)

This module defines how the system talks to the underlying LLM and how different specialist agents are configured.

```python
load_dotenv()

def _env_true(k: str) -> bool:
    return os.getenv(k, "").strip().lower() in ("1", "true", "yes", "on")

USE_OPENAI = _env_true("USE_OPENAI_LLM")
USE_OLLAMA = _env_true("USE_OLLAMA")

MODEL = os.getenv("MODEL", "gpt-4o-mini")

OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2")

def llm(temp=0.4):
    if USE_OLLAMA:
        from langchain_ollama import ChatOllama
        return ChatOllama(
            model=OLLAMA_MODEL,
            temperature=temp,
            num_ctx=4096,
            num_predict=768,
            repeat_penalty=1.1,
        )
    if USE_OPENAI:
        from langchain_openai import ChatOpenAI
        return ChatOpenAI(model=MODEL, temperature=temp)
    raise RuntimeError("No chat LLM enabled. Set USE_OLLAMA=true or USE_OPENAI_LLM=true.")
```

This variables configure which LLM backend is active and what specific model should be loaded. Then it creates and returns a configured chat model instance. Since we use Ollama: It Instantiates it with model, temperature, and extra parameters like context window size, maximum tokens to generate, and reducing repetition by setting *repeat_penalty* to 1.1.

```python
# ---- language helper ----
def _lang_instruction(answer_lang: str) -> str:

    if answer_lang == "pl":
        return """- Always answer in NATURAL POLISH.
- Use natural, idiomatic Polish culinary vocabulary.
- Avoid literal word-for-word translations from English (no calques like “gładka maszyna”; say “gładka masa”).
- Make sure every instruction step is a complete sentence that ends logically (no unfinished lines like “Dodaj wanilię i”).
- Quickly reread your answer before returning it and fix obvious grammar or wording mistakes."""
    if answer_lang == "es":
        return """- Always answer in NATURAL SPANISH.
- Use natural, idiomatic Spanish culinary vocabulary (as used in real recipes).
- Avoid literal word-for-word translations from English (no awkward calques).
- Make sure every instruction step is a complete sentence that ends logically.
- Quickly reread your answer before returning it and fix obvious grammar or wording mistakes."""
    # default: English
    return """- Always answer in NATURAL ENGLISH.
- Use clear, idiomatic cooking vocabulary (as in real cookbooks and food blogs).
- Avoid literal word-for-word translations from other languages.
- Make sure every instruction step is a complete sentence that ends logically (no half-finished lines).
- Quickly reread your answer before returning it and fix obvious grammar or wording mistakes."""
```

This function is used to define how the agent should speak in given target language, and to fix common issues like weird translations.

```python
def _polish_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Polish Chef Agent.

Context:
- You may receive user questions and tool / RAG context in Polish, Spanish, or English.

Your tasks:
- Give authentic Polish recipes, ingredients (metric), methods and key techniques.
- Briefly explain WHY important steps matter to educate the user.
- Prefer RAG passages / tool context if provided.

Language rules:
{lang_block}
- You may silently translate sources in other languages into the target language before answering.
"""
```

```python
def _peruvian_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Peruvian Chef Agent.

Context:
- You may receive user questions and tool / RAG context in Polish, Spanish, or English.

Your tasks:
- Give authentic Peruvian recipes and culinary advice, using metric units.
- Focus on sauces, ají types, citrus handling, ceviche-style preparations.
- Explain technique and food safety around raw fish and citrus where relevant.
- Prefer RAG passages / tool context if provided.

Language rules:
{lang_block}
- You may silently translate sources in other languages into the target language before answering.
"""
```

```python
def _fusion_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Fusion Coordinator Agent.

Context:
- You may receive user questions and tool / RAG context in Polish, Spanish, or English.

Your tasks:
- Combine Polish and Peruvian elements into coherent fusion dishes.
- Propose 1-2 dishes, list ingredients (metric), step-by-step method, and rationale.
- Use RAG context if available to ground your ideas.

Language rules:
{lang_block}
"""
```

```python
def _nutrition_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Nutrition Coach Agent.

Context:
- Questions and context may be in Polish, Spanish, or English.

Your tasks:
- Calculate rough macros, flag allergens, and suggest healthier swaps.
- If nutrition tool output is available, summarize it clearly.
- Be concise and educational, using metric units.

Language rules:
{lang_block}
"""
```

```python
def _quiz_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Quizmaster Agent.

Context:
- Questions and tool context may be in Polish, Spanish, or English.

Your tasks:
- Generate a 5-question quiz (mixed MCQ and short-answer) about techniques/ingredients discussed.
- Provide an answer key at the end.
- Base questions on the provided context when possible.

Language rules:
{lang_block}
"""
```

```python
def _historian_sys(answer_lang: str) -> str:
    lang_block = _lang_instruction(answer_lang)
    return f"""
You are the Culinary Historian Agent.

Context:
- Questions and documents may be in Polish, Spanish, or English.

Your tasks:
- Give 150–250 words on cultural/regional background and evolution of the dish(es).
- Use RAG sources if available; you may mention them briefly in plain text (no formal citations).
- Focus on clear, interesting storytelling.

Language rules:
{lang_block}
"""
```

Each of these functions builds a full system prompt for a specific agent role including context description, agent's tasks, the language rules (returned by *_lang_instruction*).

```python
def system_for(role: str, answer_lang: str) -> str:
    return {
        "polish": _polish_sys,
        "peruvian": _peruvian_sys,
        "fusion": _fusion_sys,
        "nutrition": _nutrition_sys,
        "quiz": _quiz_sys,
        "history": _historian_sys,
    }[role](answer_lang)
```

This part returns the correct system prompt string for a given agent role and target language. It ensures that every agents call is initialized with the correct role-specific instructions.

```python
def run_agent(
    role: str,
    user_msg: str,
    tool_context: str = "",
    answer_lang: str = "en",
) -> str:
    chain = llm()
    sys = system_for(role, answer_lang)
    messages = [SystemMessage(content=sys)]
    if tool_context:
        messages.append(AIMessage(content=f"Tool context:\n{tool_context}"))
    messages.append(HumanMessage(content=user_msg))
    resp = chain.invoke(messages)
    return resp.content
```

This provides a single, high-level function to run any of the defined agents with a user message and optional tool context.

Retrieval, Unit Conversion and Nutrition tools (tools.py)

This module has all the tools that the LangGraph can call from inside of the agents:

- a RAG search tool over the user's recipe corpus stored in a local Chroma vector database,
- a unit conversion tool for common cooking units,
- two nutrition tools using the Edamam API (single ingredients and full recipe).

```python
load_dotenv()

def _env_true(key: str) -> bool:
    return os.getenv(key, "").strip().lower() in ("1", "true", "yes", "on")

USE_NUTRITION = _env_true("USE_NUTRITION_API")

EDAMAM_APP_ID = os.getenv("EDAMAM_APP_ID")
EDAMAM_APP_KEY = os.getenv("EDAMAM_APP_KEY")
```

The purpose of this code is to:

- Help to interpret the environment variabes as booleans
- Decide whether nutrition toolss should be enabled
- Provide credentials for the Edamam nutrition API

```
def _embedder():
        from langchain_huggingface import HuggingFaceEmbeddings
        return HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

This creates a reusable embedding function for converting recipe text into vector representations, used both by *ingest.py* (for indexing) and here for retrieval.

```
def _get_vs():
    """Open the persistent Chroma collection built by ingest.py."""
    return Chroma(
        collection_name="recipes",
        embedding_function=_embedder(),
        persist_directory="memory/chroma_db",
    )
```

The purpose of this part of the code is to open the persistent Chroma vector store where ingested recipe chunks are stored.

```python
@tool("rag_search", return_direct=False)
def rag_search(query: str, k: int = 4) -> str:
    """Search the local recipe/technique corpus and return the top passages."""
    retriever = _get_vs().as_retriever(search_kwargs={"k": k})
    docs = retriever.invoke(query)
    if not docs:
        return "No relevant passages found."
    out = []
    for i, d in enumerate(docs, 1):
        out.append(f"[{i}] {d.page_content.strip()}\n(source: {d.metadata.get('source')})")
    return "\n\n".join(out)
```

This provides a RAG tool for searching the user's local recipe corpus stored in Chroma. It is padded into the agents as "Tool context", so they can ground recipes and explanations.

```python
@tool("unit_convert", return_direct=False)
def unit_convert(qty: float, unit_from: str, unit_to: str) -> str:
    """Convert between common cooking units (metric/imperial)."""
    u = unit_from.lower()
    v = unit_to.lower()
    grams_per_ounce = 28.3495
    ml_per_cup = 240.0
    if u in ["g", "gram", "grams"] and v in ["oz", "ounce", "ounces"]:
        return f"{qty} g ≈ {qty/grams_per_ounce:.2f} oz"
    if u in ["oz", "ounce", "ounces"] and v in ["g", "gram", "grams"]:
        return f"{qty} oz ≈ {qty*grams_per_ounce:.1f} g"
    if u in ["ml"] and v in ["cup", "cups"]:
        return f"{qty} ml ≈ {qty/ml_per_cup:.2f} cups"
    if u in ["cup", "cups"] and v in ["ml"]:
        return f"{qty} cups ≈ {qty*ml_per_cup:.0f} ml"
    return "Conversion not supported. Try g↔oz or ml↔cup."
```

This allows the system to convert units between metric and imperial for typical cooking scenarios.

```python
@tool("nutrition_lookup", return_direct=False)
def nutrition_lookup(ing_query: str) -> str:
    """
    Look up nutrition facts for a single ingredient using Edamam's nutrition-data endpoint.
    Enable with USE_NUTRITION_API=true and set EDAMAM_APP_ID / EDAMAM_APP_KEY in .env.
    Example ing_query: "200 g chicken breast"
    """
    if not USE_NUTRITION:
        return "Nutrition tool disabled. Set USE_NUTRITION_API=true in .env to enable."
    if not (EDAMAM_APP_ID and EDAMAM_APP_KEY):
        return "Nutrition API not configured. Set EDAMAM_APP_ID and EDAMAM_APP_KEY in .env."

    url = "https://api.edamam.com/api/nutrition-data"
    params = {"app_id": EDAMAM_APP_ID, "app_key": EDAMAM_APP_KEY, "ingr": ing_query}
    try:
        r = requests.get(url, params=params, timeout=15)
        if r.status_code != 200:
            return f"Nutrition API error: {r.status_code} — {r.text[:200]}"
        data = r.json()
        cal = data.get("calories")
        wt = data.get("totalWeight")
        return f"Query: {ing_query}\nTotal calories: {cal}\nTotal weight: {wt} g"
    except Exception as e:
        return f"Nutrition API request failed: {e}"
```

This fetches approximate calorie and weight information for a single ingredient using Edamam's *nutrition-data* endpoint.

```python
@tool("nutrition_recipe", return_direct=False)
def nutrition_recipe(ingredients: List[str]) -> str:
    """
    Nutrition for a multi-ingredient list using Edamam's nutrition-details endpoint.
    Example: ["200 g chicken breast", "1 tbsp olive oil"]
    """
    if not USE_NUTRITION:
        return "Nutrition tool disabled. Set USE_NUTRITION_API=true in .env to enable."
    if not (EDAMAM_APP_ID and EDAMAM_APP_KEY):
        return "Nutrition API not configured. Set EDAMAM_APP_ID and EDAMAM_APP_KEY in .env."

    url = "https://api.edamam.com/api/nutrition-details"
    params = {"app_id": EDAMAM_APP_ID, "app_key": EDAMAM_APP_KEY}
    payload = {"ingr": ingredients}
    try:
        r = requests.post(url, params=params, json=payload, timeout=20)
        if r.status_code != 200:
            return f"Nutrition API error: {r.status_code} — {r.text[:200]}"
        data = r.json()
        cal = data.get("calories")
        wt = data.get("totalWeight")
        return f"Ingredients: {ingredients}\nTotal calories: {cal}\nTotal weight: {wt} g"
    except Exception as e:
        return f"Nutrition API request failed: {e}"
```

This part calculates the nutrition data for an entire recipe consisting of multiple ingredients.

Orchestration Graph and Conversation Flow (graph.py)

This module defines the orchestration logic for the Chef Tutor using LangGraph.

```
class State(TypedDict, total=False):
    user_query: str
    messages: List[str]
    last_tool_output: Optional[str]
    route: Literal["polish", "peruvian", "fusion", "nutrition", "quiz", "history"]
    answer_lang: Literal["pl", "es", "en"]
```

This defines the shared structure between nodes in the LangGraph.

```
def _roll_msgs(msgs: List[str], new: str, keep: int = 6) -> List[str]:
    base = msgs or []
    base = base + [new.strip()]
    return base[-keep:]
```

This maintains a rolling window of the last few assistant messages, so it prevents the state from growing infinitely and keeps the compact history that can be used for continuity.

```
def _maybe_unit_convert(q: str) -> Optional[str]:
    m = CONV_RE.search(q)
    if not m:
        return None
    qty = float(m.group("qty"))
    u1, u2 = m.group("u1"), m.group("u2")
    try:
        return unit_convert.invoke({"qty": qty, "unit_from": u1, "unit_to": u2})
    except Exception as e:
        return f"Unit conversion failed: {e}"
```

This detects and handles unit conversion queries within a user's question.

```python
def _maybe_recipe_list(q: str) -> Optional[List[str]]:
    """Parse a semicolon/newline-separated ingredient list if present."""
    if ";" in q or "\n" in q:
        items = [s.strip() for s in re.split(r"[;\n]+", q) if s.strip()]
        return items or None
    return None
```

This parses ingredient list from user input when ingredients are separated by semicolons or new lines.

```python
def _guess_lang(text: str) -> str:
    """Detect PL / ES / EN based on input text; default to EN if detection fails."""
    try:
        code = detect(text)
    except Exception:
        return "en"
    code = code.lower()
    if code.startswith("pl"):
        return "pl"
    if code.startswith("es"):
        return "es"
    return "en"
```

Here this code detects whether the piece of text is in Polish, Spanish or English. The default is English, and this helps the graph decide in which language the assistant should answer.

```python
def _extract_answer_lang(q: str) -> str:
    """
    Decide answer language based on explicit hints or question language.

    Rules:
    - If question contains:
        * 'po angielsku' / 'in English' / 'en inglés' / 'en ingles' → 'en'
        * 'po polsku' / 'in Polish' / 'en polaco' → 'pl'
        * 'po hiszpańsku' / 'po hiszpansku' / 'in Spanish' /
          'en español' / 'en espanol' / 'en castellano' → 'es'
    - Otherwise: use detected language of the question.
    """
    q_low = q.lower()

    if (
        "po angielsku" in q_low
        or "in english" in q_low
        or "en inglés" in q_low
        or "en ingles" in q_low
    ):
        return "en"

    if "po polsku" in q_low or "in polish" in q_low or "en polaco" in q_low:
        return "pl"

    if (
        "po hiszpańsku" in q_low
        or "po hiszpansku" in q_low
        or "in spanish" in q_low
        or "en español" in q_low
        or "en espanol" in q_low
        or "en castellano" in q_low
    ):
        return "es"

    return _guess_lang(q)
```

This code decides the target answer language, based on both explicit phrases and automatic language detection. So, if user writes in Spanish, the answer will be in Spanish. If the user writes “I want this recipe in Polish”, the answer will be in Polish.

```python
def _build_user_msg_with_history(state: State) -> str:
    """
    Build the user message for the agent: short conversation history + new question.
    State["messages"] is a list of strings like "User: ...", "Assistant: ...".
    """
    history_list = state.get("messages") or []
    if not history_list:
        return state["user_query"]
    history_text = "\n".join(history_list)
    return f"Conversation so far:\n{history_text}\n\nNew question:\n{state['user_query']}"
```

This constructs a single text prompt that contains both recent dialog and the latest user question, so the agents can see the recent context.

```python
def _cleanup_output(text: str, answer_lang: str) -> str:
    """
    Optional second pass: language cleanup for PL/ES outputs.
    - For PL: fix grammar, finish sentences, remove weird calques.
    - For ES: same idea, but in Spanish.
    - For EN: return as-is.
    """
    if not text or not text.strip():
        return text

    if answer_lang == "pl":
        prompt = (
            "Popraw poniższy tekst przepisu po polsku.\n"
            "Zachowaj sens, kolejność kroków i strukturę (składniki, instrukcja), ale:\n"
            "- popraw błędy językowe i dziwne kalki z innych języków,\n"
            "- dokończ wszystkie niedokończone zdania,\n"
            "- używaj naturalnego słownictwa kulinarnego,\n"
            "- upewnij się, że tekst brzmi jak prawdziwy przepis z bloga kulinarnego.\n\n"
            f"Tekst:\n{text}"
        )
        return run_agent("polish", prompt, tool_context="", answer_lang="pl")

    if answer_lang == "es":
        prompt = (
            "Reescribe el siguiente texto de receta en ESPAÑOL NATURAL.\n"
            "Mantén el contenido y la estructura (ingredientes, pasos), pero:\n"
            "- corrige errores gramaticales y palabras raras,\n"
            "- completa frases inacabadas,\n"
            "- usa vocabulario culinario natural como en un blog de cocina.\n\n"
            f"Texto:\n{text}"
        )
        # use the Peruvian agent as a Spanish-native culinary writer
        return run_agent("peruvian", prompt, tool_context="", answer_lang="es")

    # For English, do nothing
    return text
```

This runs an optional second-pass cleanup on Polish and Spanish outputs to improve the quality of answers, to fix issues caused by local models mixing languages or creating incomplete steps in the recipes.

```python
def router(state: State) -> State:
    q = (state.get("user_query") or "")
    q_lower = q.lower()

    answer_lang = _extract_answer_lang(q)

    if "fusion" in q_lower or "combine" in q_lower:
        route = "fusion"
    elif "quiz" in q_lower:
        route = "quiz"
    elif any(k in q_lower for k in ["calorie", "macro", "protein", "kcal", "nutrition", "kcalories"]):
        route = "nutrition"
    elif any(k in q_lower for k in ["history", "culture", "origin", "background"]):
        route = "history"
    elif any(k in q_lower for k in ["peru", "peruvian", "ceviche", "ají", "aji", "anticucho", "lomo"]):
        route = "peruvian"
    elif any(k in q_lower for k in ["poland", "polish", "pierogi", "bigos", "gołąbki", "golabki", "schabowy"]):
        route = "polish"
    else:
        route = "fusion"
    return {"route": route, "answer_lang": answer_lang}
```

This part chooses which agent should be handling the current query and decides the target answer language.

```python
def use_tools(state: State) -> State:
    outs: List[str] = []
    outs.append(rag_search.invoke({"query": state["user_query"], "k": 4}))

    conv = _maybe_unit_convert(state["user_query"])
    if conv:
        outs.append(conv)

    if state.get("route") == "nutrition":
        items = _maybe_recipe_list(state["user_query"])
        if items:
            outs.append(nutrition_recipe.invoke({"ingredients": items}))
        else:
            outs.append(nutrition_lookup.invoke({"ing_query": state["user_query"]}))

    joined = "\n\n".join([o for o in outs if o and o.strip()])
    return {"last_tool_output": joined}
```

This code runs all relevant tools before the agent responds, so the agent is provided with grounded context (retrieved recipes, conversions, nutrition data) in one string.

```python
def polish_node(state: State) -> State:
    combined_msg = _build_user_msg_with_history(state)
    lang = state.get("answer_lang", "en")
    out = run_agent("polish", combined_msg, state.get("last_tool_output") or "", answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}


def peruvian_node(state: State) -> State:
    combined_msg = _build_user_msg_with_history(state)
    lang = state.get("answer_lang", "en")
    out = run_agent("peruvian", combined_msg, state.get("last_tool_output") or "", answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}


def fusion_node(state: State) -> State:
    base_ctx = state.get("last_tool_output") or ""
    combined_msg = _build_user_msg_with_history(state)
    lang = state.get("answer_lang", "en")

    pl = run_agent("polish", combined_msg, base_ctx, answer_lang=lang)
    pe = run_agent("peruvian", combined_msg, base_ctx, answer_lang=lang)

    fused_ctx = f"{base_ctx}\n\n--- Polish proposal ---\n{pl}\n\n--- Peruvian proposal ---\n{pe}"
    out = run_agent("fusion", combined_msg, fused_ctx, answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}
```

```python
def nutrition_node(state: State) -> State:
    combined_msg = _build_user_msg_with_history(state)
    lang = state.get("answer_lang", "en")
    out = run_agent("nutrition", combined_msg, state.get("last_tool_output") or "", answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}


def quiz_node(state: State) -> State:
    context = "\n\n".join((state.get("messages") or [])[-6:])
    lang = state.get("answer_lang", "en")
    out = run_agent("quiz", f"Create a 5-question quiz from this content:\n{context}", "", answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}


def history_node(state: State) -> State:
    combined_msg = _build_user_msg_with_history(state)
    lang = state.get("answer_lang", "en")
    out = run_agent("history", combined_msg, state.get("last_tool_output") or "", answer_lang=lang)
    out = _cleanup_output(out, lang)
    msgs = _roll_msgs(state.get("messages", []), out)
    return {"messages": msgs}
```

Each agent node builds a combined message with history, reads *answer_lang* from the state, calls *run_agent()*, optionally applies *_cleanup_output*, updates messages list via *_rolls_msgs*.

The *polish_node* handles Polish cuisine requests using the Polish Chef Agent prompt, in the language given by *answer_lang*. The *peruvian_node* has the same pattern but uses Peruvian Chef Agent.

The *fusion_node*:

- Uses *run_agent("polish", ...)* and *run_agent("peruvian", ...)* to generate Polish and Peruvian proposals based on the same combined message and tool context.
- Builds a *fused_ctx* containing both proposals.
- Calls *run_agent("fusion", combined_msg, fused_ctx, answer_lang=lang)* to create an actual fusion recipe using both contexts.
- Cleans up the result and updates the messages.

The *nutrition_node* uses the Nutrition Coach Agent to interpret the *last_tool_output* and answer in user-friendly way.

The *quiz-node* constructs a context string from last few messages, asks the Quizmaster Agent to create a quiz, and optionally cleans up the language.

The *history_node* calls the Culinary Historian Agent to generate cultural/ historical explanation for the dishes, using RAG context where available.

```python
def build_graph():
    g = StateGraph(State)
    g.add_node("router", router)
    g.add_node("tools", use_tools)

    g.add_node("polish", polish_node)
    g.add_node("peruvian", peruvian_node)
    g.add_node("fusion", fusion_node)
    g.add_node("nutrition", nutrition_node)
    g.add_node("quiz", quiz_node)
    g.add_node("history", history_node)

    g.add_edge("router", "tools")
    g.add_conditional_edges(
        "tools",
        lambda s: s["route"],
        {
            "polish": "polish",
            "peruvian": "peruvian",
            "fusion": "fusion",
            "nutrition": "nutrition",
            "quiz": "quiz",
            "history": "history",
        },
    )

    for node in ("polish", "peruvian", "fusion", "nutrition", "quiz", "history"):
        g.add_edge(node, END)

    g.set_entry_point("router")
    return g.compile()
```

This code defines and compiles the LangGraph that wires all nodes together.

Streamlit frontend and Chat Interface (app_streamlit.py)

This module implements the web interface for the multi-agent Chef Tutor using Streamlit. It provides:

- A sidebar where the user can upload new recipe PDF's and trigger the ingestion into the Chroma Vector database,
- A chat-style interface where the user can ask questions,

- A connection to the LangGraph orchestration and the ingestion pipeline
- Display of both the model's responses and the underlying toll context.

Metrics

```
>>> Running metrics.py…

>>> Corpus metrics
Ignoring wrong pointing object 6 0 (offset 0)
Ignoring wrong pointing object 9 0 (offset 0)
Ignoring wrong pointing object 11 0 (offset 0)
Ignoring wrong pointing object 14 0 (offset 0)
Ignoring wrong pointing object 17 0 (offset 0)
Ignoring wrong pointing object 61 0 (offset 0)
Ignoring wrong pointing object 63 0 (offset 0)
Ignoring wrong pointing object 66 0 (offset 0)
Ignoring wrong pointing object 73 0 (offset 0)
Ignoring wrong pointing object 106 0 (offset 0)
Ignoring wrong pointing object 108 0 (offset 0)
Ignoring wrong pointing object 118 0 (offset 0)
Ignoring wrong pointing object 135 0 (offset 0)
TXT files: 5
PDF files: 10
Chunks: 3156
Average chunk length: 667.1 characters
Average chunk length: 105.8 words
------------------------------------------------------------
>>> RAG metrics (Recall@4)
Recall@4: 0.0%  (0/3)
Unique sources in top-4 results: 2
Examples: ['data_pdfs/old_polish_recipes.pdf', 'data_pdfs/polish_culinary_traditions.pdf']
(.venv) juliadrygalska@Julias-MacBook-Air-2 ChefAgent 2 %
```

To understand the knowledge base behind the assistant, we computed basic corpus statistics using the same ingestion pipeline as in the application. The current dataset consists of 5 text/markdown files and 10 PDF cookbooks, which are split into 3156 chunks and stored in the Chroma vector database. The average chunk length is 667 characters (≈106 words), which provides enough context per chunk while still keeping retrieval reasonably precise.

We also performed a small retrieval experiment using three manually defined test questions (traditional cheesecake "sernik tradycyjny", pierogi ruskie and Peruvian ceviche). The current dataset consists of 5 text/markdown files and 10 PDF cookbooks, which are split into 3156 chunks and stored in the Chroma vector database. The average chunk length is 667 characters

(≈106 words), which provides enough context per chunk while still keeping retrieval reasonably precise.

Conclusion

This project started from a simple idea: we wanted a cooking assistant that actually learns from the recipes, speaks our languages (Polish, Spanish and English) and can explain what it's doing instead of just throwing random instructions at us. In the end we built a system that combines a local Llama 3.2 3B model (via Ollama), a Chroma vector database with PDFs and text recipes, and a Streamlit web app where we can chat with different "chef agents".

Right now, the assistant knows 5 text files and 10 PDF cookbooks, which after splitting into chunks gives 3156 pieces of text (about 106 words each). This is the knowledge base it uses when we ask questions. We also ran a small retrieval test (Recall@4) and got 0% on three very specific queries. At first that looks bad, but it mostly showed that the test phrases didn't match how the recipes are actually written in the PDFs. It still helped, because now we know how to evaluate retrieval properly and what to improve next (better test questions, more aligned data, maybe more recipes).

Even with a small local model, the assistant can already do quite a lot: answer in the language of the question, generate Polish and Peruvian recipes, suggest fusion ideas, estimate macros and create quizzes or short cultural notes.

For us, this is a good first prototype of a multilingual, RAG-based learning assistant. It works end-to-end, we have some basic metrics, and we clearly see the next steps: improve retrieval tests, add more and better-structured recipes, and maybe try a slightly larger model for smoother language. Overall, the project helped us connect LLMs, RAG, agents and a real UI into one working system, not just separate exercises.

AI use

Tool: OpenAI ChatGPT (GPT-5 Thinking).

Used for: planning/clarifications, drafting report & slides, and light code ideas

Not used for: data collection or final code decisions—those were done and verified by us.

Validation: all AI suggestions were tested and adjusted; our repo remained the single source of truth.